```
  function Node(storesKey)
    for char in Alphabet do
      this.children[char] ← null
    this.keyNode ← storesKey

class Trie
  #type Node
  root

  function Trie()
    root ← new Node(false)
```

As you can see, tries go beyond the usual duality between leaves and inner nodes, introducing another (orthogonal) distinction. Turns out, though, that all leaves in a well-formed, minimal trie are key nodes: a leaf would make no sense as an intermediate node, as we will see.

The fact that words are stored in paths, in fact, means that all the descendants of a node share a common prefix: the path from the root to their common parent. If, for instance, we look at figure 6.3, we can see that all nodes share the prefix "`an`", and all but one node share "`ant`". These two words, incidentally, are also stored in the trie, because the nodes at the end of their path are *key nodes*.

The root is, to all extents, an intermediate node associated with the empty string: it would be a key node only if the empty string belonged to the corpus contained in the trie.

While for spell checkers storing a Boolean in each node could be enough (after all, we only need to know if a word is in a dictionary or not), tries are often used to store or index